FLOWSTLC: An Information Flow Control Type System Based On Graded Modality

1st Zhige Chen
Dept. of Computer Science and Engineering
Southern University of Science and Technology
Shenzhen, China
12413315@mail.sustech.edu.cn

2nd Junqi Huang
Dept. of Computer Science and Engineering
Southern University of Science and Technology
Shenzhen, China
12212226@mail.sustech.edu.cn

3rd Zhiyuan Cao
Dept. of Computer Science and Engineering
Southern University of Science and Technology
Shenzhen, China
12311109@mail.sustech.edu.cn

***Abstract*—In this project, we introduce the design and implementation of a simple functional programming language with a type system that enforces secure information flow. Building on the simply-typed lambda calculus (STLC), we extend the type system with a graded modality with a security semiring. Inspired by *Graded Modal Dependent Type Theory* [1], our system statically prevents unauthorized flows of sensitive data into public outputs, ensuring the noninterference property [2].**

***Index Terms*—Computer security, information flow, noninterference, security-type systems**

I. Introduction

Modern software often handles sensitive data that must remain confidential. The *informational flow control* (or IFC) aims to ensure that high-security inputs of the program do not improperly influence low-security outputs. For example, private user information like passwords should never be leaked to public logs. Formally, the security policy of *noninterference* requires that variations in high-security inputs have no observable effect on low-security outputs. Intuitively, this means that changing a secret value should never cause any change in what an outside observer at low level sees. Violating the IFC principle can lead to information leak, so enforcing noninterference is critical for system security.

Type systems [3] [4] have been proved effective at enforcing noninterference: they associate each value with a *security label* and constrain operations so that a well-typed program is guaranteed not to leak high-security information. For example, consider the following function:

$$\text{foo} = \lambda x : \texttt{Nat}.\lambda y : \texttt{Nat}.x + y \tag{1}$$

Then such coeffect systems might allow the function to be typed as

$$\text{foo} : \texttt{Nat}^{\texttt{Sec}} \to \texttt{Nat}^{\texttt{Pub}} \to \texttt{Nat}^{\texttt{Sec}} \tag{2}$$

where the first parameter is marked as a high-security input, and the second parameter is marked as a low-security input. If a low-security result depends on a high-security input, the type system would reject the program. Thus, those security type systems can automatically verify that programs adhere to confidentiality policies.

However, existing IFC type systems can be rather inflexible or coarse-grained. Many system treat security labels in a rigid, lattice-based way [5], and often difficult to coexist with other sophisticated type system constructs. To address these issues, we propose the FLOWSTLC, a more flexible and extensible type system that track informational flows with *graded modalities*. More specifically, we will design a simply typed lambda-calculus with integrated *graded necessity* for the semiring $\{\texttt{Pub} \sqsubseteq \texttt{Sec}\}$, where $\texttt{Pub}$ represents low-security information, and $\texttt{Sec}$ represents high-security information. This system will allow us precisely control how the values interact as a type of *coeffects* [6] [7], while remain easy to extend with other program reasoning constructs like the *usage analysis* [8].

II. FLOWSTLC: The Core Calculus

A. Syntax

The FLOWSTLC is a graded extension to the simply typed lambda-calculus resembling the Fuzz type system of Reed et al. [9]. It is also similar to coeffect calculi of Brunel et al. [10] and Gaboardi et al. [11].

The syntax of FLOWSTLC is a straightforward extension of STLC with two additional constructs for introducing and eliminating the graded necessity type $\Box_\ell T$. We also add records, built-in boolean, natural numbers, and unit type to it:

$$\begin{array}{lrr} \textsf{Term}\ t & ::= & x \mid t\ t \mid \lambda x.t \mid [t] \mid \textbf{let}\ [x] = t\ \textbf{in}\ t \\ & & \{l_i = t_i^{i\in 1..n}\} \mid t.l \\ & & \texttt{true} \mid \texttt{false} \mid \textbf{if}\ t\ \textbf{then}\ t\ \textbf{else}\ t \mid n \\ & & \texttt{iszero}\ t \mid () \mid \textbf{let}\ () = t\ \textbf{in}\ t \\ \textsf{Type}\ T & ::= & T \to^{\ell} T \mid \Box_r T \\ & & \{l_i : T_i^{i\in 1..n}\} \mid \texttt{Unit} \mid \texttt{Bool} \mid \texttt{Nat} \end{array}$$

Fig. 1. Syntax of FLOWSTLC

The syntax $[t]$ promotes a term to a graded modality, and **let** $[x] = t_1$ **in** t_2 eliminate the modalities by checking whether t_2 uses t_1 w.r.t. its grade, and, if satisfied, "unbox" the modality and substitute it into t_2. The function type $T \to^{\ell} T$ records how will the function use its argument by a grade ℓ. The graded modality $\Box_{\ell}T$ is a type constructor where ℓ comes from the *security level algebra*, i.e., a semiring $\mathbb{S} = \texttt{Pub} \sqsubseteq \texttt{Sec}$. The records, projectinos, built-in booleans, natural numbers, and unit types are easy and well-studied extensions, and we will omit the explanation of these constructs here.

Typing judgements are of the regular form $\Gamma \vdash t : T$ with the typing contexts of the form:

$$\textsf{Context}\ \Gamma \ ::= \ \emptyset \mid \Gamma, x : [T]_{\ell}$$

Contexts are either empty $\emptyset$, or can be extended with a *graded assumption* $x : [T]_{\ell}$. For graded assumptions, their grades ℓ capture their substructural behavior by describing how can they be used in a term, in this case, the grade capture the security level of information. We will use $dom(\Gamma)$ to denote the variables assigned by context Γ.

B. Typing

FLOWSTLC has all three standard typing rules of STLC:

$$\frac{}{\mathbf{0} \cdot \Gamma, x : [T]_{\mathbf{1}} \vdash x : T}\ \textsc{T-Var}$$

$$\frac{\Gamma, x : [T_1]_{\ell} \vdash t : T_2}{\Gamma \vdash \lambda x : T_1.t : T_1 \to^{\ell} T_2}\ \textsc{T-Abs}$$

$$\frac{\Gamma_1 \vdash t_1 : T_{11} \to^{\ell} T_{12} \qquad \Gamma_2 \vdash t_2 : T_{11}}{\Gamma_1 + \ell \cdot \Gamma_2 \vdash t_1\ t_2 : T_{12}}\ \textsc{T-App}$$

Fig. 2. STLC rules in FLOWSTLC

The exchange rule and weakening rule that allow permutation and expansion of contexts is implicit here, instead we enables the permutation by a permutation lemma (see Permutation lemma). The differences with the standard rules are that these rules integrate grades into the premises and conclusions. For example, the T-VAR rule enforces that we only use x by requiring all other assumptions have a $\texttt{Sec}$ grade. And the abstraction rule T-ABS records the usage of variable x in function body by marking the function's type with the same grade of the assumption that assigns x. For the application rule T-APP, the concatenation combines all graded assumptions by the semiring addition $+$ (for more information see the Appendix A.3).

The next three rules build the structure of the security label semiring:

$$\frac{\Gamma \vdash t : T}{\ell \cdot \Gamma \vdash [t] : \Box_{\ell} T}\ \textsc{T-Pro}$$

$$\frac{\Gamma_1 \vdash t_1 : \Box_{\ell} T_1 \qquad \Gamma_2, x : [T_1]_{\ell} \vdash t_2 : T_2}{\Gamma_1 + \Gamma_2 \vdash \textbf{let}\ [x] = t_1\ \textbf{in}\ t_2 : T_2}\ \textsc{T-Let}$$

$$\frac{\Gamma, x : [T_2]_{\ell_1} \vdash t : T_1 \qquad \ell_2 \sqsubseteq \ell_1}{\Gamma, x : [T_2]_{\ell_2} \vdash t : T_1}\ \textsc{T-Approx}$$

Fig. 3. Grade Rules in FLOWSTLC

The *Promotion* rule T-PRO "promotes" a regular term t to a graded modality by propagating its resource requirement to the context by a *scalar multiplication* (for complete definition of scalar multiplication, see the Appendix A.3). The T-LET rule provides a way to eliminate graded modality via substitution, where graded value is "unboxed" and substituted into a graded assumption with matching grades. The context concatenation is also used in the conclusion. Finally, the *approximation* rule T-APPROX allows us to "loosen" the requirement if $\ell_2 \sqsubseteq \ell_1$.

The rules for record and projection are based on the book of Pierce and Benjamin [12]. Since FLOWSTLC has a simply-typed base, there are no subtyping rules for record types:

$$\frac{\text{for each } i,\ \Gamma \vdash t_i : T_i}{\Gamma \vdash \{l_i = t_i^{i \in 1..n}\} : \{l_i : T_i^{i \in 1..n}\}}\ \textsc{T-Record}$$

$$\frac{\Gamma \vdash t : \{l_i : T_i^{i \in 1..n}\}}{\Gamma \vdash t.l_i : T_i}\ \textsc{T-Proj}$$

Fig. 4. Record Rules in FLOWSTLC

The remaining rules specify the types of booleans, natural numbers, unit values, and conditionals, etc.

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash \texttt{true} : \texttt{Bool}}\ \textsc{T-True}$$

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash \texttt{false} : \texttt{Bool}}\ \textsc{T-False}$$

$$\frac{\Gamma_1 \vdash t_1 : \texttt{Bool} \quad \Gamma_2 \vdash t_2 : T \quad \Gamma_2 \vdash t_3 : T \quad \ell \sqsubseteq \mathbf{1}}{\ell \cdot \Gamma_1 + \Gamma_2 \vdash \textbf{if}\ t_1\ \textbf{then}\ t_2\ \textbf{else}\ t_3 : T}\ \textsc{T-Cond}$$

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash 0 : \texttt{Nat}}\ \textsc{T-Zero}$$

$$\frac{\Gamma \vdash t : \texttt{Nat}}{\Gamma \vdash \texttt{succ}\ t : \texttt{Nat}}\ \textsc{T-Succ}$$

$$\frac{\Gamma \vdash t : \texttt{Nat}}{\Gamma \vdash \texttt{pred}\ t : \texttt{Nat}}\ \textsc{T-Pred}$$

$$\frac{\Gamma \vdash t : \texttt{Nat}}{\Gamma \vdash \texttt{iszero}\ t : \texttt{Bool}}\ \textsc{T-IsZero}$$

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash () : \texttt{Unit}}\ \textsc{T-Unit}$$

$$\frac{\Gamma_1 \vdash t_1 : \texttt{Unit} \qquad \Gamma_2 \vdash t_2 : T}{\Gamma_1 + \Gamma_2 \vdash \textbf{let}\ () = t_1\ \textbf{in}\ t_2 : T}\ \textsc{T-Unit-Elim}$$

Fig. 5. Typing Rules for Built-in Booleans, Integers, etc.

A noteworthy point of these rules is the premise $\ell \sqsubseteq 1$ of rule T-COND. This requirement is added to prevent the *control flow attack* via *implicit flow*, in which the attacker uses some form of equality checking and control flow to leak information. For example, the following typing derivation is invalid in our system:

$$\frac{x : [\mathtt{Bool}]_{\mathtt{Pub}} \vdash x : \mathtt{Bool} \quad \vdash 0 : \mathtt{Nat} \quad \vdash 42 : \mathtt{Nat} \quad \mathtt{Sec} \sqsubseteq \mathbf{1}}{x : [\mathtt{Bool}]_{\mathtt{Sec}} \vdash \textbf{if } x \textbf{ then } 0 \textbf{ else } 42 : \mathtt{Nat}}$$

This is the same solution in Abel and Bernardy [13] and Choudhury et al. [14].

C. Operational Semantics

Once the type-checker shows that a program is well-typed, the AST is interpreted to execute following a standard call-by-value evaluation strategy. To make proving the type preservation theorem more easy, we specify the operational semantics of FLOWSTLC in the small-step style. We first specify the value as lambda abstractions and literals:

$$\mathsf{Value}\ v \ ::= \ \lambda x{:}\,T.t \mid n \mid \mathtt{true} \mid \mathtt{false} \mid ()$$

Fig. 6. Value of FLOWSTLC

The call-by-value reduction relation $t \to t'$ is then defined as follows:

$$\frac{t_1 \rightsquigarrow t_1'}{t_1\ t_2 \rightsquigarrow t_1'\ t_2}\ \text{E-APP1}$$

$$\frac{}{\{l_i = v_i^{i\in 1..n}\}.l_j \rightsquigarrow v_j}\ \text{E-PROJRECORD}$$

$$\frac{t \rightsquigarrow t'}{t.l \rightsquigarrow t'.l}\ \text{E-PROJ}$$

$$\frac{t_j \rightsquigarrow t_j'}{\begin{array}{l}\{l_i = v_i^{i\in 1..j-1}, l_j = t_j, l_k = t_k^{k\in j+1..n}\} \\ \rightsquigarrow \{l_i = v_i^{i\in 1..j-1}, l_j = t_j', l_k = t_k^{k\in j+1..n}\}\end{array}}\ \text{E-RECORD}$$

$$\frac{t_2 \rightsquigarrow t_2'}{v_1\ t_2 \rightsquigarrow v_1\ t_2'}\ \text{E-APP2}$$

$$\frac{}{(\lambda x : T.t)\ v \rightsquigarrow [x \mapsto v]t}\ \text{E-APPABS}$$

$$\frac{t \rightsquigarrow t'}{[t] \rightsquigarrow [t']}\ \text{E-PRO}$$

$$\frac{t_1 \rightsquigarrow t_1'}{\textbf{let } [x] = t_1 \textbf{ in } t_2 \rightsquigarrow \textbf{let } [x] = t_1' \textbf{ in } t_2}\ \text{E-LET-EVAL}$$

$$\frac{}{\textbf{let } [x] = [v] \textbf{ in } t \rightsquigarrow [x \mapsto v]t}\ \text{E-LET-UNBOX}$$

Fig. 7. Core Evaluation Rules of FLOWSTLC

The operational semantics of record and projection are specified in a standard call-by-value style:

$$\frac{}{\{l_i = v_i^{i\in 1..n}\}.l_j \rightsquigarrow v_j}\ \text{E-PROJRECORD}$$

$$\frac{t \rightsquigarrow t'}{t.l \rightsquigarrow t'.l}\ \text{E-PROJ}$$

$$\frac{t_j \rightsquigarrow t_j'}{\begin{array}{l}\{l_i = v_i^{i\in 1..j-1}, l_j = t_j, l_k = t_k^{k\in j+1..n}\} \\ \rightsquigarrow \{l_i = v_i^{i\in 1..j-1}, l_j = t_j', l_k = t_k^{k\in j+1..n}\}\end{array}}\ \text{E-RECORD}$$

Fig. 8. Evaluation Rules for Record and Projection

The operational behavior of primitive values, functions, and conditionals is then specified in a intuitive manner:

$$\frac{t_1 \rightsquigarrow t_1'}{\textbf{if } t_1 \textbf{ then } t_2 \textbf{ else } t_3 \rightsquigarrow \textbf{if } t_1' \textbf{ then } t_2 \textbf{ else } t_3}\ \text{E-IF-EVAL}$$

$$\frac{}{\textbf{if } \mathtt{true} \textbf{ then } t_2 \textbf{ else } t_3 \rightsquigarrow t_2}\ \text{E-IF-TRUE}$$

$$\frac{}{\textbf{if } \mathtt{false} \textbf{ then } t_2 \textbf{ else } t_3 \rightsquigarrow t_3}\ \text{E-IF-FALSE}$$

$$\frac{t_1 \rightsquigarrow t_1'}{\textbf{let } () = t_1 \textbf{ in } t_2 \rightsquigarrow \textbf{let } () = t_1' \textbf{ in } t_2}\ \text{E-UNIT-ELIM-EVAL}$$

$$\frac{}{\textbf{let } () = () \textbf{ in } t \rightsquigarrow t}\ \text{E-UNIT-ELIM}$$

$$\frac{t \rightsquigarrow t'}{\mathtt{succ}\ t \rightsquigarrow \mathtt{succ}\ t'}\ \text{E-SUCC}$$

$$\frac{t \rightsquigarrow t'}{\mathtt{pred}\ t \rightsquigarrow \mathtt{pred}\ t'}\ \text{E-PRED}$$

$$\frac{t \rightsquigarrow t'}{\mathtt{iszero}\ t \rightsquigarrow \mathtt{iszero}\ t'}\ \text{E-ISZERO}$$

$$\frac{}{\mathtt{pred}\ 0 \rightsquigarrow 0}\ \text{E-PRED-ZERO}$$

$$\frac{}{\mathtt{pred}\ \mathtt{succ}\ t \rightsquigarrow t}\ \text{E-PRED-SUCC}$$

$$\frac{}{\mathtt{iszero}\ 0 \rightsquigarrow \mathtt{true}}\ \text{E-ISZERO-ZERO}$$

$$\frac{}{\mathtt{iszero}\ \mathtt{succ}\ t \rightsquigarrow \mathtt{false}}\ \text{E-ISZERO-SUCC}$$

Fig. 9. Evaluation Rules for Built-in Booleans, Integers, etc.

D. Graded Modality

The central innovation of FLOWSTLC is the use of a graded modality $\Box_\ell T$ to track and control information flow. This modality is parameterized by a security grade ℓ drawn from the semiring $\mathbb{S}$, where the ordering signifies that $\mathtt{Sec}$ information is more sensitive than $\mathtt{Pub}$.

Intuitively, a term of type $\Box_\ell T$ is a value that should be used according to some security policies, the grade ℓ attached to the modality acts as a capability or a requirement: it specifies the minimum security level of the context necessary to "unbox" and use the value.

The power of this approach lies in the algebraic structure of the security semiring. The semiring operations (meet for addition, join for multiplication) naturally model the combination of security constraints:

- **Context Concatenation** uses the join operation ($\sqcap$) to combine graded assumptions for the same variable. This reflects the principle that if a value can be used publicly, them it should also can be used secretly elsewhere. For example, using a variable in two different parts of a program, one requiring Sec and the other Pub, results in a combined requirement of Pub ($\texttt{Sec} \sqcap \texttt{Pub} = \texttt{Pub}$).
- **Context Scalar Multiplication** uses the meet operation ($\sqcup$). to strengthen the security requirements of a context. This is crucial in the T-PRO rule for ensuring that a promoted term does not leak its contents to a lower grade.

In essence, the graded modality $\Box_\ell T$ serves as a security-aware container. The type system's rules ensure that the "capability" ℓ needed to open this container is always respected, thereby statically guaranteeing that high-sensitivity data cannot flow into low-sensitivity outputs.

E. A Simple Example

We demonstrate the system's capability of enforcing noninterferencee by showing the system would reject the following program:

$$\lambda x : \Box_{\texttt{Sec}} T.\textbf{let}\ [y] = x\ \textbf{in}\ y : \Box_{\texttt{Sec}} T \to^{\texttt{Pub}} T$$

since it try to use a Sec value to compute a Pub value. To type this term, we first need to show that

$$x : [\Box_{\texttt{Sec}} T]_{\texttt{Pub}} \vdash \textbf{let}\ [y] = x\ \textbf{in}\ y : \Box_{\texttt{Sec}} T \to^{\texttt{Pub}} T$$

then according to the premises of T-LET rule, we need to show the following

1) $x : [\Box_{\texttt{Sec}} T]_{\texttt{Pub}} \vdash x : \Box_{\texttt{Sec}} T$
2) $y : [T]_{\texttt{Sec}} \vdash y : T$

The (1) is trivial to show, but there is no rule that allows the derivation of (2). T-VAR rule doesn't help here because it requires that the used variable must have a Pub grade. So the term is ill-typed.

III. METATHEORY

A. Type Safety

Lemma 1 (Permutation). *If $\Gamma \vdash t : T$ and Δ is a permutation of Γ, then $\Delta \vdash t : T$ and the derivation depth of the latter is the same as the former.*

The proof for the permutation lemma is trivial since assumptions in contexts are unrelated in a simply-typed context. Then we prove the well-typedness of substitution, this lemma is then used to establish syntactic type safety.

Lemma 2 (Well-typed Substitution). *If $\Gamma_1 \vdash t_1 : T_1$ and $\Gamma_2, x : [T_1]_\ell \vdash t_2 : T_2$, then we have $\Gamma_1 + \ell \cdot \Gamma_2 \vdash [x \mapsto t_1]t_2 : T_2$.*

We then state the type safety lemma as follows:

Theorem 1 (Type Preservation). *If $\Gamma \vdash t : T$, then either t is a value or there exists a t' s.t. $t \rightsquigarrow t'$ with $\Gamma \vdash t : T$.*

B. Strong Normalization

Theorem 2 (Strong Normalization). *If t is a closed and well-typed FlowSTLC term, then t is normalizable.*

IV. FUNDAMENTAL THEOREMS AND NONINTERFERENCE

Our definition of type semantics by building a logical relation model following the work of Rajani et al. [15]. The key difference between our approach and the work of Rajani et al. is that our calculus use a coeffect-based analysis instead of effect-based analysis.

We first define a type-indexed unary relation $[\![T]\!]_\mathcal{V}$ which captures the set of irreducible terms that inhabit type T. The relation is formally defined as a set of mutually inductive definitions:

$$\begin{aligned}
[\![T]\!]_\mathcal{E} &= \{t \mid t \rightsquigarrow^* v \implies v \in [\![T]\!]_\mathcal{V}\} \\
[\![\texttt{Unit}]\!]_\mathcal{V} &= \{()\} \\
[\![\texttt{Bool}]\!]_\mathcal{V} &= \{\texttt{true}, \texttt{false}, \texttt{iszero}\ n\} \\
[\![\texttt{Nat}]\!]_\mathcal{V} &= \{n \mid n \in \mathbb{N}\} \\
[\![T_1 \to^\ell T_2]\!]_\mathcal{V} &= \{\lambda x.t \mid \forall v.[v] \in [\![\Box_\ell T_1]\!]_\mathcal{E} \implies \\
&\qquad [x \mapsto v]t \in [\![T_2]\!]_\mathcal{E}\} \\
[\![\Box_\ell T]\!]_\mathcal{V} &= \{[t] \mid t \rightsquigarrow^* v \implies v \in [\![T]\!]_\mathcal{E}\} \\
[\![\{l_i : T_i^{i\in 1..n}\}]\!]_\mathcal{V} &= \{\{l_i = t_i^{i\in 1..n}\} \mid \forall i.t_i \rightsquigarrow^* v_i \implies v_i \in [\![T_i]\!]_\mathcal{E}\} \\
[\![\Gamma]\!] &= \{\gamma \mid x : [T]_\ell \in \Gamma \wedge [\gamma(x)] \in [\![\Box_\ell T]\!]_\mathcal{E}\}
\end{aligned}$$

Fig. 10. Unary value, expression, and context relations

For the function type $T_1 \to^\ell T_2$, the value inhabiting its interpretation are abstractions $\lambda x.t$ where the substitution of value v into the body t is in the interpretation of the expression relation at the result type. The interpretation of the graded modal type $\Box_\ell T$ is the set of values which are the promotions of terms in the interpretation T. Finally, the $[\![\Gamma]\!]$ relation defines an interpretation of the context Γ as a set of maps from variables to expressions (denoted by γ) that can be substituted for each variable.

We also define a binary relation which captures *indistinguishability* from the perspective of an adversary level $adv \in \mathbb{S}$. The relation is formally defined by the following figure:

$$
\begin{aligned}
[\![T]\!]^{adv}_{\mathcal{E}} &= \{(t_1, t_2) \mid t_1 \rightsquigarrow^* v_1 \wedge t_2 \rightsquigarrow^* v_2 \implies \\
&\qquad (v_1, v_2) \in [\![T]\!]^{adv}_{\mathcal{V}}\} \\
[\![\texttt{Unit}]\!]^{adv}_{\mathcal{V}} &= \{((), ())\} \\
[\![\texttt{Bool}]\!]^{adv}_{\mathcal{V}} &= \{(b, b) \mid b \in \{\texttt{true}, \texttt{false}, \texttt{iszero}\, n\}\} \\
[\![\texttt{Nat}]\!]^{adv}_{\mathcal{V}} &= \{(n, n) \mid n \in \mathbb{N}\} \\
[\![T_1 \to^{\ell} T_2]\!]^{adv}_{\mathcal{V}} &= \{(\lambda x.t_1, \lambda x.t_2) \mid \\
&\quad (\forall v, v'.([v],[v']) \in [\![\Box_\ell T_1]\!]^{adv}_{\mathcal{E}} \\
&\quad\quad \implies ((x \mapsto v)t_1, (x \mapsto v')t_2) \in [\![T_2]\!]^{adv}_{\mathcal{E}}) \\
&\quad \wedge (\forall v.[v] \in [\![\Box_\ell T_1]\!]_{\mathcal{E}} \implies \\
&\quad\quad [x \mapsto v]t_1 \in [\![T_2]\!]_{\mathcal{E}}) \\
&\quad \wedge (\forall v.[v] \in [\![\Box_\ell T_1]\!]_{\mathcal{E}} \implies \\
&\quad\quad [x \mapsto v]t_2 \in [\![T_2]\!]_{\mathcal{E}})\} \\
[\![\Box_\ell T]\!]^{adv}_{\mathcal{V}} &= \begin{cases} \{([v_1, v_2]) \mid (v_1, v_2) \in [\![\Box_\ell T]\!]^{adv}_{\mathcal{E}}\} \\ \quad \ell \leq adv \\ \{([v_1, v_2]) \mid v_1 \in [\![\Box_\ell T]\!]_{\mathcal{E}} \wedge v_2 \in [\![\Box_\ell T]\!]_{\mathcal{E}}\} \\ \quad \neg \ell \leq adv \end{cases} \\
\left[\!\!\left[\{l_i : T_i^{i \in 1..n}\}\right]\!\!\right]^{adv}_{\mathcal{V}} &= \{(\{l_i = t_i^{i \in 1..n}\}, \{l_i = t_i'^{i \in 1..n}\}) \mid \\
&\quad \forall i. t_i \rightsquigarrow^* v_i \wedge t_i' \rightsquigarrow^* v_i' \implies \\
&\quad (v_i, v_i') \in [\![T_i]\!]^{adv}_{\mathcal{E}}\} \\
[\![\Gamma]\!]^{adv} &= \{(\gamma_1, \gamma_2) \mid x : [T]_\ell \in \Gamma \\
&\quad \wedge [\gamma_1(x), \gamma_2(x)] \in [\![\Box_\ell T]\!]^{adv}_{\mathcal{E}}\}
\end{aligned}
$$

Fig. 11. Binary value, expression, and context relations

The binary relation specified here only capture the concept *termination-insensitive noninterference* [16], since we are only interested in the relation of the evaluation results. To make it termination sensitve, we need to also prove the co-termination of the two terms in the binary relation. Given the limited time for this project, we leave this as a future work direction.

For the interpretation of the function type $T_1 \to^{\ell} T_2$, the binary relation relates two abstractions where when two arbitrary values of $\Box_\ell T_1$ are substituted into their bodies, the substitution results would relate at the result type T_2. The most important interpretation here is the one for the graded type $\Box_\ell T$. For the graded types, if the graded ℓ is less or equal to the adversary level adv, then t_1 and t_2 must be related at the type T. Otherwise, t_1 and t_2 only need to be in their corresponding unary value interpretation and thus cannot be distinguished. This precisely captures the intuition that a low level adversary (i.e. $adv = \texttt{Pub}$) cannot distinguish two secret values (i.e. $\ell = \texttt{Sec}$). Finally, the interpretation of contexts $[\![\Gamma]\!]^{adv}$ reuses the interpretation for graded types to capture the grading.

A. Fundamental Theorems and Noninterference Property

To formalize the noninterference property for our type system, we need a so-called "fundamental theorem" that connects the syntactic typing with semantic interpretation. We define a unary and a binary version of the theorem for the two types of relations above respectively.

Theorem 3 (Unary Fundamental Theorem)**.** *For all well-typed terms $\Gamma \vdash t : T$, if $\gamma \in [\![\Gamma]\!]$, then we have $\gamma\ t \in [\![T]\!]_{\mathcal{E}}$. Where $\gamma\ t$ represents the result of applying the subtitution of the mapping from variables to terms given by γ.*

The theorem can be intuitively understood as, if t is a term of type T under a context Γ, and γ is a unary substitution in the interpretation of Γ. Then we can say that $\gamma\ t$ is in the unary relation at the type T.

However, since the binary relation involves an external observer, we need additional structure to formally specify it. This needed structure is a subclass of semirings which we call *noninterfering semirings*:

Definition 1 (Noninterfering Semiring)**.** *A noninterfering semiring $\mathcal{R}$ is a pre-ordered semring with the following additional axioms:*

- *Weak idempotence of multiplication: $r \cdot r \leq r$*
- *Antisymmetry: $(r \leq s) \wedge (s \leq r) \implies r = s$*
- *Multiplicative unit is minimal: $1 \leq r$*
- *Additive unit is maximal: $r \leq 0$*

where r, s are two elements of the semiring $\mathcal{R}$.

Proposition 1 (Properties of Noninterfering Semiring)**.** *For a noninterfering semiring and two elements r_1, r_2, s in it, we have the following properties:*

- *Decreasing addition: if $r_1 \leq r_2$, then $r_1 + s \leq r_2$.*
- *Increasing multiplication: if $r_1 \leq r_2$, then $r_1 \leq s \cdot r_2$ and $r_1 \leq r_2 \cdot s$.*

We can also derive the minimality of 1 and the maximality of 0 from these two properties respectively.

Proposition 2 (Security Semiring is Noninterfering)**.** *The induced semiring $\mathbb{S}$ is noninterfering, with*

- *Weak idempotence of multiplication trivially follows the property of $\sqcup$.*
- *Antisymmetry follows from the ordering.*
- *Minimality of 1 and the maximality of 0 follows from the definition of $\mathbb{S}$.*

Now we can formally give the binary version of the fundamental theorem on noninterfering semirings:

Theorem 4 (Binary Fundamental Theorem)**.** *For all well-typed terms $\Gamma \vdash t : T$ with a semiring $\mathcal{R}$ parameterizing the calculus that is* noninterfering*, if $(\gamma_1, \gamma_2) \in [\![\Gamma]\!]^{adv}$, then we have $(\gamma_1\ t, \gamma_2\ t) \in [\![T]\!]^{adv}_{\mathcal{E}}$. Where $\gamma_1\ t$ and $\gamma_2\ t$ represents the result of applying the multi-substitution of the mapping from variables to terms given by γ_1 and γ_2.*

With the fundamental theorem established, we can now state the final theorem of noninterference:

Theorem 5 (Noninterfernce of FLOWSTLC)**.** *For all judgements $x : [T_1]_r \vdash t : \Box_{adv}\mathbb{B}$ where $adv \leq r$ and $adv \neq r$, then given $\emptyset \vdash v_1 : T$ and $\emptyset \vdash v_2 : T$ then $[x \mapsto v_1]t \equiv [x \mapsto v_2]t$. Where $\equiv$ is the full β-equality of terms and $\mathbb{B}$ denotes an arbitrary base type.*

For a complete proof for the fundamental theorem and the noninterference theorem, please refer to the Appendix B.

V. Bidirectional Type Checking Algorithm

So far we have only specified the typing system of FLOWSTLC in a declarative manner, but for actual implementation we need an algorithmic system. In this chapter we introduce the bidirectional typing for FLOWSTLC. The typing rules are defined by two mutually recursive functions for *checking* and *synthesis*, of the form:

$$\begin{array}{ll} (\textit{checking}) & \Phi \vdash t \Leftarrow T \dashv \Delta \\ (\textit{synthesis}) & \Phi \vdash t \Rightarrow T \dashv \Delta \end{array} \tag{3}$$

Here we following a similar approach to Hodas [17] and Polakow [18]. We split the context into two parts:

- Type context Φ: maps variables to types, this is the input to the algorithm.
- Usage context Δ: records exactly the variables used in t and their computed grades, this is the output of the algorithm.

To support the semiring structure of $\mathbb{S}$, we define the following pointwise operations on the usage context Δ:

Definition 2 (Operations on Usage Contexts)**.**

- *Zero* 0*: maps all variables to* $\mathbf{0}$*.*
- *Addition* $\Delta_1 + \Delta_2$*: returns* Δ *where* $\Delta(x) = \Delta_1(x) + \Delta_2(x)$*.*
- *Scaling* $r \cdot \Delta$*: returns* Δ *where* $\Delta(x) = r \cdot \Delta_1(x)$*.*
- *Joining* $\Delta_1 \cup \Delta_2$*: returns* Δ *where* $\Delta(x) = \Delta_1(x) \sqcup \Delta_2(x)$*.*

Now we introduce the important rules here. For a complete list of the bidirectional typing rules, please refer to appendix.

The variable rule generates a usage of $\mathbf{1}$ for the variable used and $\mathbf{0}$ for everything else:

$$\frac{\Phi(x) = T}{\Phi \vdash x \Leftarrow T \dashv \{x : \mathbf{1}\}} \text{ BD-VAR}$$

We can swith from synthesis to checking seamlessly:

$$\frac{\Phi \vdash t \Rightarrow T_1 \dashv \Delta \qquad T_1 = T_2}{\Phi \vdash t \Leftarrow T_2 \dashv \Delta} \text{ BD-SWITCH}$$

For function abstractions, we check the function body against its return type. The critical check here is that the calculated usage of the bound variable x must $\leq$ the declared grade r on the arrow:

$$\frac{\Phi, x : T_1 \vdash t \Leftarrow T_2 \qquad \Delta(x) \leq r}{\Phi \vdash \lambda x : T_1 . t \Leftarrow T_1 \to^r T_2 \dashv \Delta \setminus \{x\}} \text{ BD-ABS}$$

The function application synthesizes the function type, checks type of the argument, then scales the argument's usage by the function's input grade r:

$$\frac{\Phi \vdash t_1 \Rightarrow T_1 \to^r T_2 \dashv \Delta_1 \qquad \Phi \vdash t_2 \Leftarrow T_1 \dashv \Delta_2}{\Phi \vdash t_1\ t_2 \Rightarrow T_2 \dashv \Delta_1 + (r \cdot \Delta_2)} \text{ BD-APP}$$

For the graded modalities, the promotion rule scales the entire usage context of the underlying term by r:

$$\frac{\Phi \vdash t \Leftarrow T \dashv \Delta}{\Phi \vdash [t] \Leftarrow \Box_r T \dashv r \cdot \Delta} \text{ BD-PRO}$$

Modality elimination synthesizes the modality type $\Box_r T$ and then checks the body. The rule ensures that the usage of the unboxed variable inside the body fits within the grade r provided by the modality type:

$$\frac{\Phi \vdash t_1 \Rightarrow \Box_r T \dashv \Delta_1 \quad \Phi, x : T \vdash t_2 \Leftarrow T_2 \dashv \Delta_2 \quad \Delta_2(x) \leq r}{\Phi \vdash \textbf{let } [x] = t_1 \textbf{ in } t_2 \Leftarrow T_2 \dashv \Delta_1 + (\Delta_2 \setminus \{x\})}$$

For the conditional, we check the types of the condition and both branches. The usage contexts of the branches is joined to ensure we account for the worst-case usage of either execution path. The rule also requires that the guard usage must be scaled by $r \leq \mathbf{1}$ to prevent control-flow attacks:

$$\frac{\Phi \vdash t \Leftarrow \texttt{Bool} \dashv \Delta_1 \quad \Phi \vdash t_1 \Leftarrow T \dashv \Delta_2 \quad \Phi \vdash t_2 \Leftarrow T \dashv \Delta_3}{\Phi \vdash \textbf{if } t \textbf{ then } t_1 \textbf{ else } t_2 \Leftarrow T \dashv \Delta_1 + \Delta_2 + \Delta_3}$$

The bidirectional rules for records, natural numbers, booleans, and unit values are trivial to specify and we omit them here.

VI. Implementation and Examples

VII. Related Work

Language-based IFC has a rich literature. Sabelfeld and Myers [3] survey a variety of security type systems that enforce noninterference via typing. Seminal work by Volpano et al. [16] introduced a static type system for a simple imperative language, ensuring well-typed programs satisfy noninterference. Extensions include JFlow [19] and JIF [20] for Java, which associate security labels with Java types to track flows, and FlowCaml [21] for OCaml, which similarly adds a security type-checker. Generally, these systems label each variable as high or low and enforce rules so that no operation can use a high value in a low context.

More recently, graded type theories have been proposed to generalize such analyses. Graded type systems annotate types with additional information ("grades") to capture various properties of programs. For example, the Granule language [8] uses graded modalities to track the effects and coeffects of program. In information flow use-cases, types are graded by a security lattice, allowing automatic enforcement of noninterference. Moon et al. introduce the Graded Modal Dependent Type Theory (GRTT) [1], which extends dependent type system with graded modality and show that the grades can be effective at reasoning of programs. This work demonstrates that graded modality can capture fine-grained flow policies in types. Similarly, Marshall and Orchard 22 demonstrated a graded-modal framework that can enforce confidentiality and even integrity simultaneously.

Other approaches include dynamic IFC (e.g. LIO monad in Haskell [23]) or hybrid systems, but our focus is purely static. In summary, while classical security type sytems (JIF, FlowCaml, etc.) enforce noninterference via fixed lattice-labels. Our project draws on these ideas: we adapt graded modalities to a simple functional language to track information flows more flexibly.

VIII. Further Work and Conclusion

In summary, FLOWSTLC has demonstrated that a simple lambda calculus with graded necessity can enforce secure information flow. By instantiating grades with the security semiring $\mathbb{S}$, our system statically tracks data labels at the type level. And with typing rules that enforcing the noninterference property, we guarantee that well-typed FLOWSTLC programs cannot leak secrets.

Despite these positive results, FLOWSTLC in its current form is an *idealized model* with clear limitations. For one, the core calculus is deliberately small and omits many practical language features. It has only base types and functions (no records, no algebraic data types, no recursive types, etc.), and it does not include polymorphism or dependent typing. As observed in prior work, static IFC models with minimal features have historically been considered ”too limited or too restrictive to be used in practice” [19]. Thus, our model inherits this limitation of expressiveness. Second, FLOWSTLC exists only as a theoretical calculus and has not been integrated into a full-blown programming language. We consider that bridging this gap is a non-trivial challenge. For example, a recent survey noted that even in a rich language like Rust, implementing a sound IFC system requires ”building an ad-hoc effect tracking system using bleeding-edge features of the Rust compiler”. Finally, we have not evaluated performance or conducted real-world case studies. There are no benchmarks on typing or runtime costs, nor have we applied FlowSTLC to any non-trivial programs. In contrast, systems like JFlow and Cocoon [24] have shown that static IFC checking can incur negligible overhead. Without such evaluation, the practical impact of FLOWSTLC remains speculative.

Given these limitations, there are many promising directions for future work. We highlight several concrete directions:

- **Extending the type system with dependent types and polymorphism.** One natural extension is to enrich FLOWSTLC with more expressive typing disciplines. In particular, adding *dependent types* could allow security policies to depend on program values and thus encode more precise confidentiality and declassification properties. Prior work (e.g. DepSec for Idris language [25]) shows that dependent types increase the expressiveness of static IFCs. Similarly, introducing *parametric polymorphism* would permit writing generic secure functions without duplicating code.
- **Combining graded modalities with effect, usage, or resource typing.** Another interesting direction is to study how the security grading interacts with other static analyses. Graded type theories generalize both effect systems (via graded monads) and coeffect/usage systems (via graded comonads). For example, FLOWSTLC might be extended with an effect system that tracks side-effects or I/O on secret data (similar to Koka's effect typing system [26] [27]). Alternatively, one could explore *coeffect-like* analyses where the context is graded alongside security. Integrating graded security labels with *resource-* or *capability-aware typing* could yield richer guarantees.
- **Embedding FLOWSTLC in a practical language or compiler.** To bring theory closer to practice, a implementation of FlowSTLC's typing discipline in a real programming environment is essential. One approach is to create a domain-specific language (DSL) or library for an existing language (e.g. Haskell or OCaml) that enforces these graded security types. Recent work (e.g. [24]) shows that it is possible to add IFC to Rust without modifying the compiler. Similarly, the Granule project [8] demonstrates that a language with graded, linear, and dependent types can be realized in practice. Following these examples, we could adapt FLOWSTLC's type checker into an implementation (for instance, a GHC plugin) to test on larger codebases. Such an embedding would enable empirical measurement of type-checking performance and runtime costs. It would also allow exploration of practical issues (language interoperability, tooling integration, etc.) that are vital for any real applications of our outcome.

In conclusion, FLOWSTLC provides a formal foundation for secure information flow via graded modal types, but many challenges remain. Future work will pursue richer type features, deeper integration with program reasoning (effects, resources), and practical implementation efforts. We are optimistic that bridging these gaps will bring secure flow typing closer to real-world programming. By pursuing these directions, we hope to develop a mature framework that combines the rigorous guarantees of our type-theoretic approach with the expressiveness and practicality needed for deployed systems.

Acknowledgment

We would like to express our sincere gratitude to the Theoretical Computer Science StackExchange user taquetgauche, who reviewed our early prototype type systems and provided valuable feedback on how to proceed.

We are also deeply thankful to all members of the Theoretical Computer Science Society of SUSTech for their continued support and for the many insightful ideas they shared with us throughout this project.

References

[1] B. Moon, H. Eades III, and D. Orchard, “Graded modal dependent type theory,” in *European Symposium on Programming*. Springer International Publishing Cham, 2021, pp. 462–490.

[2] G. Smith, “Principles of secure information flow analysis,” in *Malware Detection*. Springer, 2007, pp. 291–307.

[3] A. Sabelfeld and A. C. Myers, “Language-based information-flow security,” *IEEE Journal on selected areas in communications*, vol. 21, no. 1, pp. 5–19, 2003.

[4] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1999, pp. 147–160.

[5] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
[6] T. Petricek, D. Orchard, and A. Mycroft, "Coeffects: Unified static analysis of context-dependence," in *International Colloquium on Automata, Languages, and Programming*. Springer, 2013, pp. 385–397.
[7] ——, "Coeffects: a calculus of context-dependent computation," *ACM SIGPLAN Notices*, vol. 49, no. 9, pp. 123–135, 2014.
[8] D. Orchard, V.-B. Liepelt, and H. Eades III, "Quantitative program reasoning with graded modal types," *Proceedings of the ACM on Programming Languages*, vol. 3, no. ICFP, pp. 1–30, 2019.
[9] J. Reed, M. Gaboardi, and B. Pierce, "Distance makes the types grow stronger: A calculus for differential privacy (extended version)."
[10] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, "A core quantitative coeffect calculus," in *European Symposium on Programming Languages and Systems*. Springer, 2014, pp. 351–370.
[11] M. Gaboardi, S.-y. Katsumata, D. Orchard, F. Breuvart, and T. Uustalu, "Combining effects and coeffects via grading," *ACM SIGPLAN Notices*, vol. 51, no. 9, pp. 476–489, 2016.
[12] B. C. Pierce, *Types and programming languages*. MIT press, 2002.
[13] A. Abel and J.-P. Bernardy, "A unified view of modalities in type systems," *Proceedings of the ACM on Programming Languages*, vol. 4, no. ICFP, pp. 1–28, 2020.
[14] P. Choudhury, H. Eades III, R. A. Eisenberg, and S. Weirich, "A graded dependent type system with a usage-aware semantics," *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, pp. 1–32, 2021.
[15] V. Rajani and D. Garg, "Types for information flow control: Labeling granularity and semantic models," in *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, 2018, pp. 233–246.
[16] D. Volpano, C. Irvine, and G. Smith, "A sound type system for secure flow analysis," *Journal of computer security*, vol. 4, no. 2-3, pp. 167–187, 1996.
[17] J. S. Hodas, *Logic programming in intuitionistic linear logic: theory, design, and implementation*. University of Pennsylvania, 1994.
[18] J. Polakow, "Embedding a full linear lambda calculus in haskell," *ACM SIGPLAN Notices*, vol. 50, no. 12, pp. 177–188, 2015.
[19] A. C. Myers, "Jflow: Practical mostly-static information flow control," in *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1999, pp. 228–241.
[20] K. Pullicino, "Jif: Language-based information-flow security in java," *arXiv preprint arXiv:1412.8639*, 2014.
[21] V. Simonet and I. Rocquencourt, "Flow caml in a nutshell," in *Proceedings of the first APPSEM-II workshop*, 2003, pp. 152–165.
[22] D. Marshall and D. Orchard, "Graded modal types for integrity and confidentiality," *arXiv preprint arXiv:2309.04324*, 2023.
[23] D. Stefan, A. Russo, J. C. Mitchell, and D. Mazières, "Flexible dynamic information flow control in haskell," in *Proceedings of the 4th ACM Symposium on Haskell*, 2011, pp. 95–106.
[24] A. Lamba, M. Taylor, V. Beardsley, J. Bambeck, M. D. Bond, and Z. Lin, "Cocoon: Static information flow control in rust," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 166–193, 2024.
[25] S. Gregersen, S. E. Thomsen, and A. Askarov, "A dependently typed library for static information-flow control in idris," in *International Conference on Principles of Security and Trust*. Springer, 2019, pp. 51–75.
[26] D. Leijen, "Koka: Programming with row polymorphic effect types," *arXiv preprint arXiv:1406.2061*, 2014.
[27] ——, "Algebraic effects for functional programming," *Technical Report. MSR-TR-2016–29. Microsoft Research technical report, Tech. Rep.*, 2016.

APPENDIX

A. Complete Specification of FLOWSTLC

1) Syntax:

Term	t	$::=$	x	*variable*
		$\mid$	$t\ t$	*application*
		$\mid$	$\lambda x.t$	*abstraction*
		$\mid$	$[t]$	*packing*
		$\mid$	$\textbf{let}\ [x] = t\colon T\ \textbf{in}\ t$	*unpacking*
		$\mid$	$\{l_i = t_i^{i\in 1..n}\}$	*record*
		$\mid$	$t.l$	*projection*
		$\mid$	$\texttt{true}$	*true constant*
		$\mid$	$\texttt{false}$	*false constant*
		$\mid$	$\textbf{if}\ t\ \textbf{then}\ t\ \textbf{else}\ t$	*conditional*
		$\mid$	n	*natural number*
		$\mid$	$\texttt{iszero}\ t$	*zero test*
		$\mid$	$()$	*unit*
		$\mid$	$\textbf{let}\ () = t\ \textbf{in}\ t$	*unit elimination*
Number	n	$::=$	0	*zero constant*
		$\mid$	$\texttt{succ}\ n$	*successor*
		$\mid$	$\texttt{pred}\ n$	*predecessor*
Type	T	$::=$	$\mathbb{B}$	*base type*
		$\mid$	$\{l_i\colon T_i^{i\in 1..n}\}$	*record type*
		$\mid$	$T \to^{\ell} T$	*function type*
		$\mid$	$\Box_r T$	*graded modality*
Base type	$\mathbb{B}$	$::=$	$\texttt{Unit}$	*unit type*
		$\mid$	$\texttt{Nat}$	*natural number type*
		$\mid$	$\texttt{Bool}$	*boolean type*
Value	v	$::=$	$\lambda x\colon T.t$	*abstraction value*
		$\mid$	$\{l_i = v_i^{i\in 1..n}\}$	*record value*
		$\mid$	n	*natural number value*
		$\mid$	$\texttt{true}$	*true value*
		$\mid$	$\texttt{false}$	*false value*
		$\mid$	$()$	*unit value*
Context	Γ	$::=$	$\emptyset$	*empty context*
		$\mid$	$\Gamma, x\colon [T]_r$	*graded assumption*
Security	ℓ	$::=$	$\texttt{Sec}$	*high-security label*
		$\mid$	$\texttt{Pub}$	*low-security label*

2) Security Level Semiring:

Definition 3 (Security Level Semiring)**.** *The security level semiring $\mathbb{S}$ is a two-point lattice of security levels $\mathbb{S} = \{\texttt{Pub} \sqsubseteq \texttt{Sec}\}$ with*

- ***0*** $= \texttt{Sec}$
- ***1*** $= \texttt{Pub}$
- *Addition as the meet:* $r + s = r \sqcap s$
- *Multiplication as the join* $r \cdot s = r \sqcup s$

See Appendix B for a proof that this algebra is a indeed a semiring.

3) Auxiliary Definitions:

Definition 4 (Context Concatenation)**.**

$$\emptyset + \Gamma = \Gamma$$
$$\Gamma + \emptyset = \Gamma$$
$$(\Gamma, x : [T]_r) + (\Gamma', x : [T]_s) = (\Gamma + \Gamma'), x : [T]_{r+s}$$

Definition 5 (Context Scalar Multiplication)**.**

$$r \cdot \emptyset = \emptyset$$
$$r \cdot (\Gamma, x : [T]_s) = (r \cdot \Gamma), x : [T]_{(r \cdot s)}$$

4) Typing Rules:

$$\frac{}{\mathbf{0} \cdot \Gamma, x : [T]_{\mathbf{1}} \vdash x : T}\ \text{T-VAR}$$

$$\frac{\Gamma, x : [T_1]_\ell \vdash t : T_2}{\Gamma \vdash \lambda x : T_1.t : T_1 \to^\ell T_2}\ \text{T-ABS}$$

$$\frac{\Gamma_1 \vdash t_1 : T_{11} \to^\ell T_{12} \qquad \Gamma_2 \vdash t_2 : T_{11}}{\Gamma_1 + \ell \cdot \Gamma_2 \vdash t_1\ t_2 : T_{12}}\ \text{T-APP}$$

$$\frac{\Gamma \vdash t : T}{\ell \cdot \Gamma \vdash [t] : \Box_\ell T}\ \text{T-PRO}$$

$$\frac{\Gamma_1 \vdash t_1 : \Box_\ell T_1 \qquad \Gamma_2, x : [T_1]_\ell \vdash t_2 : T_2}{\Gamma_1 + \Gamma_2 \vdash \textbf{let}\ [x] = t_1\ \textbf{in}\ t_2 : T_2}\ \text{T-LET}$$

$$\frac{\Gamma, x : [T_2]_{\ell_1} \vdash t : T_1 \qquad \ell_2 \sqsubseteq \ell_1}{\Gamma, x : [T_2]_{\ell_2} \vdash t : T_1}\ \text{T-APPROX}$$

$$\frac{\text{for each } i,\ \Gamma \vdash t_i : T_i}{\Gamma \vdash \{l_i = t_i^{i \in 1..n}\} : \{l_i : T_i^{i \in 1..n}\}}\ \text{T-RECORD}$$

$$\frac{\Gamma \vdash t : \{l_i : T_i^{i \in 1..n}\}}{\Gamma \vdash t.l_i : T_i}\ \text{T-PROJ}$$

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash \mathtt{true} : \mathtt{Bool}}\ \text{T-TRUE}$$

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash \mathtt{false} : \mathtt{Bool}}\ \text{T-FALSE}$$

$$\frac{\Gamma_1 \vdash t_1 : \mathtt{Bool} \quad \Gamma_2 \vdash t_2 : T \quad \Gamma_2 \vdash t_3 : T \quad \ell \sqsubseteq \mathbf{1}}{\ell \cdot \Gamma_1 + \Gamma_2 \vdash \textbf{if}\ t_1\ \textbf{then}\ t_2\ \textbf{else}\ t_3 : T}\ \text{T-COND}$$

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash 0 : \mathtt{Nat}}\ \text{T-ZERO}$$

$$\frac{\Gamma \vdash t : \mathtt{Nat}}{\Gamma \vdash \mathtt{succ}\ t : \mathtt{Nat}}\ \text{T-SUCC}$$

$$\frac{\Gamma \vdash t : \mathtt{Nat}}{\Gamma \vdash \mathtt{pred}\ t : \mathtt{Nat}}\ \text{T-PRED}$$

$$\frac{\Gamma \vdash t : \mathtt{Nat}}{\Gamma \vdash \mathtt{iszero}\ t : \mathtt{Bool}}\ \text{T-ISZERO}$$

$$\frac{}{\mathbf{0} \cdot \Gamma \vdash () : \mathtt{Unit}}\ \text{T-UNIT}$$

$$\frac{\Gamma_1 \vdash t_1 : \mathtt{Unit} \qquad \Gamma_2 \vdash t_2 : T}{\Gamma_1 + \Gamma_2 \vdash \textbf{let}\ () = t_1\ \textbf{in}\ t_2 : T}\ \text{T-UNIT-ELIM}$$

5) Evaluation Rules:

$$\frac{t_1 \rightsquigarrow t_1'}{t_1\ t_2 \rightsquigarrow t_1'\ t_2}\ \text{E-APP1}$$

$$\frac{t_2 \rightsquigarrow t_2'}{v_1\ t_2 \rightsquigarrow v_1\ t_2'}\ \text{E-APP2}$$

$$\frac{}{(\lambda x : T.t)\ v \rightsquigarrow [x \mapsto v]t}\ \text{E-APPABS}$$

$$\frac{t \rightsquigarrow t'}{[t] \rightsquigarrow [t']}\ \text{E-PRO}$$

$$\frac{t_1 \rightsquigarrow t_1'}{\textbf{let}\ [x] = t_1\ \textbf{in}\ t_2 \rightsquigarrow \textbf{let}\ [x] = t_1'\ \textbf{in}\ t_2}\ \text{E-LET-EVAL}$$

$$\frac{}{\textbf{let}\ [x] = [v]\ \textbf{in}\ t \rightsquigarrow [x \mapsto v]t}\ \text{E-LET-UNBOX}$$

$$\frac{}{\{l_i = v_i^{i \in 1..n}\}.l_j \rightsquigarrow v_j}\ \text{E-PROJRECORD}$$

$$\frac{t \rightsquigarrow t'}{t.l \rightsquigarrow t'.l}\ \text{E-PROJ}$$

$$\frac{t_j \rightsquigarrow t_j'}{\begin{array}{c}\{l_i = v_i^{i \in 1..j-1}, l_j = t_j, l_k = t_k^{k \in j+1..n}\} \\ \rightsquigarrow \{l_i = v_i^{i \in 1..j-1}, l_j = t_j', l_k = t_k^{k \in j+1..n}\}\end{array}}\ \text{E-RECORD}$$

$$\frac{t_1 \rightsquigarrow t_1'}{\textbf{if}\ t_1\ \textbf{then}\ t_2\ \textbf{else}\ t_3 \rightsquigarrow \textbf{if}\ t_1'\ \textbf{then}\ t_2\ \textbf{else}\ t_3}\ \text{E-IF-EVAL}$$

$$\frac{}{\textbf{if}\ \mathtt{true}\ \textbf{then}\ t_2\ \textbf{else}\ t_3 \rightsquigarrow t_2}\ \text{E-IF-TRUE}$$

$$\frac{}{\textbf{if}\ \mathtt{false}\ \textbf{then}\ t_2\ \textbf{else}\ t_3 \rightsquigarrow t_3}\ \text{E-IF-FALSE}$$

$$\frac{t_1 \rightsquigarrow t_1'}{\textbf{let}\ () = t_1\ \textbf{in}\ t_2 \rightsquigarrow \textbf{let}\ () = t_1'\ \textbf{in}\ t_2}\ \text{E-UNIT-ELIM-EVAL}$$

$$\frac{}{\textbf{let}\ () = ()\ \textbf{in}\ t \rightsquigarrow t}\ \text{E-UNIT-ELIM}$$

$$\frac{t \rightsquigarrow t'}{\mathtt{succ}\ t \rightsquigarrow \mathtt{succ}\ t'}\ \text{E-SUCC}$$

$$\frac{t \rightsquigarrow t'}{\mathtt{pred}\ t \rightsquigarrow \mathtt{pred}\ t'}\ \text{E-PRED}$$

$$\frac{t \rightsquigarrow t'}{\mathtt{iszero}\ t \rightsquigarrow \mathtt{iszero}\ t'}\ \text{E-ISZERO}$$

$$\frac{}{\mathtt{pred}\ 0 \rightsquigarrow 0}\ \text{E-PRED-ZERO}$$

$$\frac{}{\mathtt{pred}\ \mathtt{succ}\ t \rightsquigarrow t}\ \text{E-PRED-SUCC}$$

$$\frac{}{\mathtt{iszero}\ 0 \rightsquigarrow \mathtt{true}}\ \text{E-ISZERO-ZERO}$$

$$\frac{}{\mathtt{iszero}\ \mathtt{succ}\ t \rightsquigarrow \mathtt{false}}\ \text{E-ISZERO-SUCC}$$

6) Bidirectional Typing:

$$\frac{\Phi \vdash t \Rightarrow T_1 \dashv \Delta \qquad T_1 = T_2}{\Phi \vdash t \Leftarrow T_2 \dashv \Delta}\ \textsc{BD-Switch}$$

$$\frac{\Phi(x) = T}{\Phi \vdash x \Leftarrow T \dashv \{x : \mathbf{1}\}}\ \textsc{BD-Var}$$

$$\frac{\Phi, x : T_1 \vdash t \Leftarrow T_2 \qquad \Delta(x) \leq r}{\Phi \vdash \lambda x : T_1 . t \Leftarrow T_1 \to^r T_2 \dashv \Delta \setminus \{x\}}\ \textsc{BD-Abs}$$

$$\frac{\Phi \vdash t_1 \Rightarrow T_1 \to^r T_2 \dashv \Delta_1 \qquad \Phi \vdash t_2 \Leftarrow T_1 \dashv \Delta_2}{\Phi \vdash t_1\ t_2 \Rightarrow T_2 \dashv \Delta_1 + (r \cdot \Delta_2)}\ \textsc{BD-App}$$

$$\frac{\Phi \vdash t \Leftarrow T \dashv \Delta}{\Phi \vdash [t] \Leftarrow \Box_r T \dashv r \cdot \Delta}\ \textsc{BD-Pro}$$

$$\frac{\Phi \vdash t_1 \Rightarrow \Box_r T \dashv \Delta_1 \quad \Phi, x : T \vdash t_2 \Leftarrow T_2 \dashv \Delta_2 \quad \Delta_2(x) \leq r}{\Phi \vdash \textbf{let } [x] = t_1 \textbf{ in } t_2 \Leftarrow T_2 \dashv \Delta_1 + (\Delta_2 \setminus \{x\})}\ \textsc{BD-Let}$$

$$\frac{\text{for each } i,\ \Phi \vdash t_i \Leftarrow T_i \dashv \Delta_i}{\Phi \vdash \{l_i = t_i^{i \in 1..n}\} \Leftarrow \{l_i : T_i^{i \in 1..n}\} \dashv \sum_{i=1}^n \Delta_i}\ \textsc{BD-Record}$$

$$\frac{\Phi \vdash t \Rightarrow \{l_i : T_i^{i \in 1..n}\} \dashv \Delta}{\Phi \vdash t.l_i \Rightarrow T_i \dashv \Delta}\ \textsc{BD-Proj}$$

$$\frac{}{\Phi \vdash \texttt{true} \Rightarrow \texttt{Bool} \dashv \emptyset}\ \textsc{BD-True}$$

$$\frac{}{\Phi \vdash \texttt{false} \Rightarrow \texttt{Bool} \dashv \emptyset}\ \textsc{BD-False}$$

$$\frac{\Phi \vdash t \Leftarrow \texttt{Bool} \dashv \Delta_1 \quad \Phi \vdash t_1 \Leftarrow T \dashv \Delta_2 \quad \Phi \vdash t_2 \Leftarrow T \dashv \Delta_3}{\Phi \vdash \textbf{if } t \textbf{ then } t_1 \textbf{ else } t_2 \Leftarrow T \dashv \Delta_1 + \Delta_2 + \Delta_3}\ \textsc{BD-Cond}$$

$$\frac{}{\Phi \vdash 0 \Rightarrow \texttt{Nat} \dashv \emptyset}\ \textsc{BD-Zero}$$

$$\frac{\Phi \vdash t \Leftarrow \texttt{Nat} \dashv \Delta}{\Phi \vdash \texttt{succ}\ t \Rightarrow \texttt{Nat} \dashv \Delta}\ \textsc{BD-Succ}$$

$$\frac{\Phi \vdash t \Leftarrow \texttt{Nat} \dashv \Delta}{\Phi \vdash \texttt{pred}\ t \Rightarrow \texttt{Nat} \dashv \Delta}\ \textsc{BD-Pred}$$

$$\frac{\Phi \vdash t \Leftarrow \texttt{Nat} \dashv \Delta}{\Phi \vdash \texttt{iszero}\ t \Rightarrow \texttt{Bool} \dashv \Delta}\ \textsc{BD-IsZero}$$

$$\frac{}{\Phi \vdash () \Rightarrow \texttt{Unit} \dashv \emptyset}\ \textsc{BD-Unit}$$

$$\frac{\Phi \vdash t_1 \Leftarrow \texttt{Unit} \dashv \Delta_1 \qquad \Phi \vdash t_2 \Leftarrow T \dashv \Delta_2}{\Phi \vdash \textbf{let } () = () \textbf{ in } t \Leftarrow T \dashv \Delta_1 + \Delta_2}\ \textsc{BD-Unit-Elim}$$

B. Proofs

1) Security Level Semiring:

Proof.

- **Associativity of addition**: $(a+b)+c = a+(b+c)$
  This is trivial since the semiring addition is meet, and join is associative in a lattice.
- **Commutativity of addition**: $a+b = b+a$
  This is also trivial since meet is commutative in a lattice.
- **Additive identity**: $a + \mathbf{0} = a$ for all a
  Since $0 = \texttt{Sec}$, and $\texttt{Sec}$ be the maximum in $\mathbb{S}$, we have
  $$\begin{aligned} \texttt{Sec} \sqcap \texttt{Sec} &= \texttt{Sec} \\ \texttt{Pub} \sqcap \texttt{Sec} &= \texttt{Pub} \end{aligned} \tag{4}$$
- **Associativity of multiplication**: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
  This is trivial by the associativity of join operation.
- **Multiplication distributes over addition**: $a \cdot (b+c) = (a \cdot b) + (a \cdot c)$ and $(a+b) \cdot c = (a \cdot c) + (b \cdot c)$
  The distributivity trivially holds since $\mathbb{S}$ is a two-point lattice.
- **Multiplicative identity**: $a \cdot \mathbf{1} = a$ for all a
  Since $1 = \texttt{Pub}$ and $\texttt{Pub} \sqsubset \texttt{Sec}$, we have
  $$\begin{aligned} \texttt{Pub} \sqcup \texttt{Pub} &= \texttt{Pub} \\ \texttt{Pub} \sqcup \texttt{Sec} &= \texttt{Sec} \end{aligned} \tag{5}$$
- **Multiplication by 0 annihilates**: $\mathbf{0} \cdot a = a \cdot \mathbf{0} = \mathbf{0}$ for all a
  Since $0 = \texttt{Sec}$ and $\texttt{Pub} \sqsubset \texttt{Sec}$, we have
  $$\begin{aligned} \texttt{Sec} \sqcup \texttt{Sec} &= \texttt{Sec} \\ \texttt{Pub} \sqcup \texttt{Sec} &= \texttt{Sec} \end{aligned} \tag{6}$$

Thus the security level algebra is a semiring. □

2) Unary Fundamental Theorem:

Proof. By induction on the typing derivation of $\Gamma \vdash t : T$

- *Case* T-VAR: $\mathbf{0} \cdot \Gamma, x : [T]_{\mathbf{1}} \vdash x : T$.
  Let $\gamma \in [\![\mathbf{0} \cdot \Gamma, x : [T]_{\mathbf{1}}]\!]$. By the definition of the unary relation, γ maps x s.t. $[\![\gamma(x)]\!] \in [\![\Box_{\mathbf{1}} T]\!]_{\mathcal{E}}$. The unary relation for graded modalities $[\![\Box_{\mathbf{1}} T]\!]_{\mathcal{E}}$ ignores the grade r and contains terms that, when unboxed, are in $[\![T]\!]_{\mathcal{E}}$. Therefore, $\gamma(x) \in [\![T]\!]_{\mathcal{E}}$. Since the term is just x, $\gamma\ t = \gamma(x)$, satisfying the goal.
- *Case* T-ABS: $\Gamma \vdash \lambda x : T_1.t : T_1 \to^r T_2$ derived from $\Gamma, x : [T_1]_r \vdash t : T_2$.
  Assume $\gamma \in [\![\Gamma]\!]$, we must show that $\gamma(\lambda x : T_1.t) \in [\![T_1 \to^r T_2]\!]_{\mathcal{E}}$. Since $\lambda x : T_1.t$ is a value, substituting v into the body yields an expression in $[\![T_2]\!]_{\mathcal{E}}$. Let $v \in [\![\Box_r T_1]\!]_{\mathcal{E}}$. Construct the extended substitution $\gamma' = \gamma, x \mapsto v$. By definition, $\gamma' \in [\![\Gamma, x : [T_1]_r]\!]$. By the induction hypothesis on the premise $\Gamma, x : [T_1]_r \vdash t : T_2$, we have $\gamma'\ t \in [\![T_2]\!]_{\mathcal{E}}$. Since $\gamma'\ t = [x \mapsto v](\gamma\ t)$, the condition holds.
- *Case* T-APP: $\Gamma_1 + r \cdot \Gamma_2 \vdash t_1\ t_2 : T_2$.
  Assume $\gamma \in [\![\Gamma_1 + r \cdot \Gamma_2]\!]$. By the definition of context addition and scalar multiplication, γ can be viewed as

covering the requirements of both Γ_1 and $r \cdot \Gamma_2$. By the induction hypothesis on t_1, $\gamma\ t_1 \in [\![T_1 \to^r T_2]\!]_{\mathcal{E}}$. This implies $\gamma\ t_1$ reduces to a value $\lambda x : T_1.t$ in $[\![T_1 \to^r T_2]\!]_{\mathcal{V}}$. By the induction hypothesis on t_2, $\gamma\ t_2 \in [\![T_1]\!]_{\mathcal{E}}$. The definition of the function value relation states that applying such a function to an argument v (where $[v] \in [\![\Box_r T_1]\!]_{\mathcal{E}}$) yields a result in $[\![T_2]\!]_{\mathcal{E}}$. Since $\gamma\ t_2 \in [\![T_1]\!]_{\mathcal{E}}$, its corresponding modality $[\gamma\ t_2]$ is in $[\![\Box_r T_1]\!]_{\mathcal{E}}$. Thus, $(\gamma\ t_1)\ (\gamma\ t_2) \in [\![T_2]\!]_{\mathcal{E}}$.

- *Case* T-PRO: $r \cdot \Gamma \vdash [t] : \Box_r T$.
  By induction hypothesis, $\gamma\ t \in [\![T]\!]_{\mathcal{E}}$. By the definition, the unary relation $[\![\Box_r T]\!]_{\mathcal{V}}$ consists of terms $[u]$ where $[u] \in [\![T]\!]_{\mathcal{E}}$. Thus, we have $[\gamma\ t] \in [\![\Box_r T]\!]_c\, alv$, so $\gamma\ [t] \in [\![\Box_r T]\!]_{\mathcal{E}}$.
- *Case* T-LET: $\Gamma_1 + \Gamma_2 \vdash \textbf{let}\ [x] = t_1\ \textbf{in}\ t_2 : T_2$.
  By the induction hypothesis, $\gamma\ t_1 \in [\![\Box_r T_1]\!]_{\mathcal{E}}$, so $\gamma\ t_1 \rightsquigarrow^* [v]$ with $v \in [\![T_1]\!]_{\mathcal{E}}$. By the induction hypothesis on t_2 with $\gamma[x \mapsto v]$, we have $\gamma'\ t_2 \in [\![T_2]\!]_{\mathcal{E}}$.
- *Case* T-APPROX: $\Gamma, x : [T_1]_s : t : T_2$ where $s \leq r$.
  The unary relation for $\Box_r T$ does not depend on the value of r. Thus, if $\gamma(x) \in [\![\Box_s T_1]\!]_{\mathcal{E}}$, it is also in $[\![\Box_r T_1]\!]_{\mathcal{E}}$. The induction hypothesis applies directly.
- *Case* T-RECORD: $\Gamma \vdash \{l_i = t_i^{i \in 1..n} : \{l_i : T_i^{i \in 1..n}\}\}$.
  By the induction hypothesis, for all $i \in 1..n$, $\gamma\ t_i \in [\![T_i]\!]_{\mathcal{E}}$. Thus, $\{l_i = t_i^{i \in 1..n}\} \in [\![\{l_i : T_i^{i \in 1..n}\}]\!]_{\mathcal{V}}$.
- *Case* T-PROJ: $\Gamma \vdash t.l_i : T_i$.
  By the induction hypothesis, $\gamma\ t \rightsquigarrow^* \{l_i = v_i^{i \in 1..n}\}$ where $v_i \in [\![T_i]\!]_{\mathcal{V}}$. So $t.l_i \in [\![T_i]\!]_{\mathcal{V}}$.
- *Case* T-TRUE: $\mathbf{0} \cdot \Gamma \vdash \texttt{true} : \texttt{Bool}$.
  Trivial since $\texttt{true} \in [\![\texttt{Bool}]\!]_{\mathcal{V}}$ by definition.
- *Case* T-FALSE: $\mathbf{0} \cdot \Gamma \vdash \texttt{false} : \texttt{Bool}$.
  Trivial since $\texttt{false} \in [\![\texttt{Bool}]\!]_{\mathcal{V}}$ by definition.
- *Case* T-COND: $r \cdot \Gamma_1 + \Gamma_2 \vdash \textbf{if}\ t_1\ \textbf{then}\ t_2\ \textbf{else}\ t_3 : T_1$.
  By the induction hypothesis, $\gamma\ t_1 \in [\![\texttt{Bool}]\!]_{\mathcal{E}}$. If $\gamma\ t_1 \rightsquigarrow^* \texttt{true}$, we use the induction hypothesis on t_2, which gives $\gamma\ t_2 \in [\![T_1]\!]_{\mathcal{E}}$, so the result is in $[\![T_1]\!]_{\mathcal{E}}$. If $\gamma\ t_1 \rightsquigarrow^* \texttt{false}$, we use the induction hypothesis on t_3, which also results in $[\![T_1]\!]_{\mathcal{E}}$.
- *Case* T-ZERO: $\mathbf{0} \cdot \Gamma \vdash 0 : \texttt{Nat}$.
  Trivial since $0 \in [\![\texttt{Nat}]\!]_{\mathcal{V}}$ by definition.
- *Case* T-SUCC: $\mathbf{0} \cdot \Gamma \vdash \texttt{succ}\ n : \texttt{Nat}$.
  Trivial since $\texttt{succ}\ n \in [\![\texttt{Nat}]\!]_{\mathcal{V}}$ by definition.
- *Case* T-PRED: $\mathbf{0} \cdot \Gamma \vdash \texttt{pred}\ n : \texttt{Nat}$.
  Trivial since $\texttt{pred}\ n \in [\![\texttt{Nat}]\!]_{\mathcal{V}}$ by definition.
- *Case* T-ISZERO: $\mathbf{0} \cdot \Gamma \vdash \texttt{iszero}\ n : \texttt{Bool}$.
  Trivial since $\texttt{iszero}\ n \in [\![\texttt{Bool}]\!]_{\mathcal{V}}$ by definition.
- *Case* T-UNIT: $\mathbf{0} \cdot \Gamma \vdash () : \texttt{Unit}$.
  Trivial since $() \in [\![\texttt{Unit}]\!]_{\mathcal{V}}$ by definition.
- *Case* T-UNIT-ELIM: $\Gamma_1 + \Gamma_2 \vdash \textbf{let}\ () = t_1\ \textbf{in}\ t_2 : T_2$.
  We have $\gamma\ t_1 \rightsquigarrow^* ()$. Applying the induction hypothesis to t_2 yields the result in $[\![T_2]\!]_{\mathcal{E}}$.

□

3) Binary Fundamental Theorem:

Proof. By induction on the typing derivation.

- *Case* T-VAR:
- *Case* T-ABS:
- *Case* T-APP:
- *Case* T-PRO:
- *Case* T-LET:
- *Case* T-APPROX:
- *Case* T-RECORD:
- *Case* T-PROJ:
- *Case* T-TRUE:
- *Case* T-FALSE:
- *Case* T-COND:
- *Case* T-ZERO:
- *Case* T-SUCC:
- *Case* T-PRED:
- *Case* T-ISZERO:
- *Case* T-UNIT:
- *Case* T-UNIT-ELIM:

□

4) Noninterference of FLOWSTLC*:*

Proof. We need to substitute v_1 and v_2 for x. So we must show that the pair of environments $(\{x \mapsto v_1\}, \{x \mapsto v_2\})$ is in the binary context relation $[\![x : [T]_s]\!]^{adv}$. This requires us to show $([v_1], [v_2]) \in [\![\Box_s T]\!]_{\mathcal{E}}^{adv}$.

Given $adv \leq s$ and $adv \neq s$, we check if $s \leq adv$ (is the input observable?). If $s \leq adv$, combined with $adv \leq s$ and the antisymmmetry axiom of the noninterfering semiring, we would have $s = adv$. But the premise states that $adv \neq s$. Therefore we have $\neg(s \leq adv)$.

From $\neg(s \leq adv)$, the definition of the graded modality relation $[\![\Box_s T]\!]_{\mathcal{E}}^{adv}$ simplifies to the unary check $\{([u_1], [u_2]) \mid u_1 \in [\![[T]]\!]_{\mathcal{E}} \wedge u_2 \in [\![[T]]\!]_{\mathcal{E}}\}$. By the Unary Fundamental Theorem. since v_1 and v_2 are well-typed terms, they are in $[\![[]\ T]_{\mathcal{E}}$. Thus, the inputs are validly related in the context $(\gamma_1, \gamma_2) \in [\![x : [T]_s]\!]^{adv}$.

By the Binary Fundamental Theorem, the results are related at the output type: $([x \mapsto v_1]t, [x \mapsto v_2]) \in [\![\Box_{adv} \mathbb{B}]\!]_{\mathcal{E}}^{adv}$. The output has a grade adv and clearly by reflexivity $adv \leq adv$. Therefore, the relation uses the observable case $\{([u_1], [u_2]) \mid (u_1, u_2) \in [\![\mathbb{B}]\!]_{\mathcal{E}}^{adv}\}$. This implies the unboxed terms u_1, u_2 are related in $[\![\mathbb{B}]\!]_{\mathcal{E}}^{adv}$.

By the definition of the binary relation at base types, the terms reduce to identical values, so the equality $[x \mapsto v_1]t \equiv [x \mapsto v_2]t$ holds. □